

MLFQ — Multilevel Feedback Queue

One CPU, whole ticks, CPU-only processes; no switching cost. Service is the total CPU work needed to finish.

Q0 has highest priority; Q2 has lowest. Each queue has two budgets:

- **Quantum (time slice):** CPU ticks allowed in one turn.
- **Allotment:** total CPU ticks allowed at that priority before demotion.

Defaults: quantum and allotment [2, 4, 8]; boost interval 10. All settings are positive whole numbers.

Rules

1. New arrivals join Q0's tail with zero usage. Simultaneous arrivals keep input order.
2. Run the head of the highest-priority nonempty queue. Use Round Robin within each queue.
3. Higher-priority work preempts the runner: return it to its queue's **front**, preserving both counters. Same-priority arrivals do not preempt.
4. Each running tick: remaining service $\text{--} 1$; quantum usage $\text{+} 1$; allotment usage $\text{+} 1$. Waiting consumes neither budget.
5. **Completion comes first** at the next boundary, including when expiry or boost coincides.
6. Allotment exhausted: reset both counters and move the unfinished process to the next lower queue's tail (Q2 $\rightarrow$ Q1). This can occur midway through a quantum.
7. Only quantum expired: move to the same queue's tail; reset quantum usage and keep allotment usage.
8. Boost at positive multiples of the interval, never at time 0. Collect waiting processes in Q0, Q1, Q2 order, preserving each queue's order. Stop and append any ongoing runner **last**. Move everyone collected to Q0, reset both counters, and preserve remaining service.
9. Dispatch normally. A lone process can immediately run again. Idle if nothing is ready; stop when all processes finish.

Events at the same boundary

Without a boost: completion $\rightarrow$ arrivals $\rightarrow$ requeue expired work $\rightarrow$ higher-priority preemption $\rightarrow$ dispatch.

With a boost: completion $\rightarrow$ requeue expired work $\rightarrow$ boost $\rightarrow$ arrivals $\rightarrow$ dispatch.

Allotment exhaustion takes precedence over quantum expiry. A process requeued before a boost takes its queue position; only an ongoing runner is appended last.

Examples

Separate budgets: With quantum 1 and allotment 4, a ten-tick process gets four one-tick turns before demotion; six service ticks remain (assuming no boost). With quantum 2 and allotment 3, demotion occurs one tick into the second turn.

Boost during a turn: A runs in Q1; B, C wait in Q1; D waits in Q2. Boost gives Q0 [B, C, D, A]. B runs next.

Expiry with boost: A exhausts its Q0 allotment; B waits in Q0, C in Q1, D in Q2. Requeue A behind C, then boost: Q0 [B, C, A, D].